

Desenvolvimento Orientado a Tipos em Python

O que é o projeto

vídeo: <https://youtu.be/d6C8kBUIM2c?si=cSDnXQJCP5-MDRb2>

Esse projeto é uma coleção de técnicas que fui aprendendo durante a disciplina e tentando aplicar em Python. A ideia não é reproduzir o sistema de tipos de Haskell, mas ver até onde conceitos parecidos podem ser representados com os recursos do Python.

A principal ferramenta usada foi o mypy, um verificador estático de tipos que analisa o código sem executá-lo e aponta usos incompatíveis com os tipos declarados.

Algumas implementações não representam exatamente os conceitos teóricos vistos em Haskell, mas chegam a efeitos práticos parecidos. Quando isso acontece, deixo essa diferença indicada no próprio projeto, como no exemplo de DataKinds.

Ao executar `main.py` (alternativamente, consultando `docs/output.pdf`), passamos por todas as técnicas, com explicações, exemplos práticos e a execução do próprio mypy em cada caso.

Como surgiu a ideia

Eu trabalho frequentemente com Python há cerca de três anos. Durante as aulas de tipos e haskell, várias vezes pensei: “acho que consigo fazer algo semelhante em Python”

Desde as primeiras aulas, eu ia implementando pequenos experimentos tentando aproximar, dentro das limitações do sistema de tipos de Python, conceitos estudados na disciplina, como phantom types, DataKinds, restrições de tipos, tipos dependentes e etc.

Como utilizar

O projeto usa **Python 3.14+** e gerenciador de projeto [uv](#). Na raiz do repositório:

```
uv sync
uv run python main.py
```

Ao executar `main.py`, as técnicas são apresentadas com explicações, exemplos e a execução do `mypy`. Alguns erros são intencionais e fazem parte das demonstrações.

Caso não seja possível executar o projeto, a saída completa está disponível em `docs/output.pdf`